Shahbaz Khalid

RAG chunking

Semantic chunking

Cut where the meaning jumps between sentences. The defaults cut one-topic documents too.

INSIDE

- How it picks where to cut
- Real distances from a real run
- Fixes for the defaults that bite
- Is it worth the embedding cost?

Cheat sheet at the end

Shahbaz Khalid

Cut where the meaning jumps

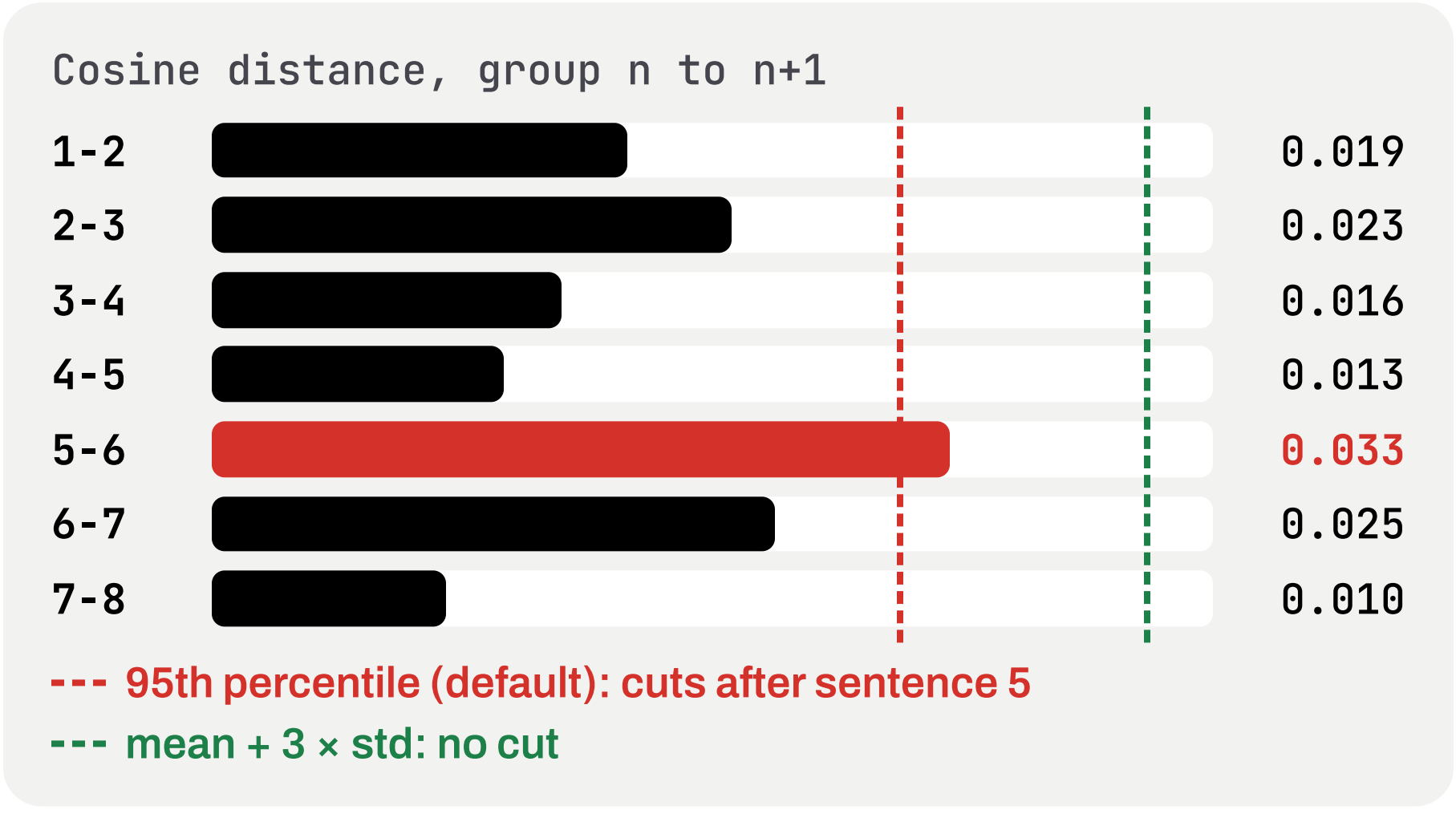
Recursive splitting cuts on separators: paragraph breaks, line breaks, spaces. Semantic chunking compares neighbouring sentences and cuts where they stop being similar. LangChain's `SemanticChunker` does it in 5 steps:

1. Split the text into sentences with a regex.
2. Group each sentence with its neighbours (`buffer_size=1`: one before, one after).
3. Embed every group.
4. Measure the cosine distance from each group to the next.
5. Cut where the distance is above a threshold: by default, the 95th percentile of all distances.

Shahbaz Khalid

The default cuts the top 5%

8 sentences, one topic: refunds. The distance between each sentence group and the next, from a real run:



A percentile is relative, so the biggest gaps get cut whether or not the topic changes. Embeddings in this run: spaCy's en_core_web_md vectors.

Shahbaz Khalid

Choose what counts as a jump

`breakpoint_threshold_type` sets the cut-off. The same 8 refund sentences, each type at its default amount:

`"percentile"` default, 95 — ✕ Top 5% of gaps

2 chunks: cut after sentence 5

`"gradient"` 95 — ✕ Top 5% of distance changes

2 chunks: cut after sentence 4

`"standard_deviation"` 3 — ✓ Mean + 3 × std

1 chunk

`"interquartile"` 1.5 — ✓ Mean + 1.5 × IQR

1 chunk

`number_of_chunks` sets the threshold that gives about that many chunks. Use it only when you know how many sections a document has.

Shahbaz Khalid

No size cap

`SemanticChunker` has `min_chunk_size` but no maximum. 24 refund sentences, one topic, `standard_deviation`:

24 sentences, 1,227 characters

Chunk 1

> Refunds are accepted within 30 days of purchase. After 30 days, refunds need manager approval. … Appeals are reviewed by a team lead within 3 business days.

✕ The whole document comes back as one chunk

Then a 500-character `RecursiveCharacterTextSplitter`

Chunk 1
465

> Refunds are accepted within 30 days… Shipping fees are refunded when the item arrived damaged.

Chunk 2
479

> Customers attach a photo when they report damage… Duplicate charges are refunded in full without approval.

Chunk 3
281

> Chargebacks pause any open refund request… Appeals are reviewed by a team lead within 3 business days.

✓ 3 chunks under the cap, each ending on a full sentence

Shahbaz Khalid

Sentences come from a regex

The default `sentence_split_regex` is `(?⩽[.?!])\s+`: a cut after every . ? or ! followed by whitespace.

```
"For refunds over $500, e.g. laptops, ask Dr. Patel.
She approves them within 2 days."
```

Sentence 1	For refunds over $500, **e.g.**
	---------- Split inside a sentence ----------
Sentence 2	laptops, ask **Dr.**
	---------- Split inside a sentence ----------
Sentence 3	**Patel.**
Sentence 4	She approves them within 2 days.

- Text with no . ? or ! comes back as one sentence, so there's nothing to compare.
- Pass a `sentence_split_regex` that fits your text.

Shahbaz Khalid

It embeds every sentence group

Finding the cuts takes one embedding per sentence group, before a single chunk is indexed. Measured on the 24-sentence document:

Counted by wrapping the embeddings object

```
sentences          24
embedding inputs   24
text               1,227 chars
embedded           3,551 chars  (2.9x)
```

- Then the chunks get embedded again for search.
- Every re-ingest pays for both passes.
- With a hosted embedding model, that's about 3x the text in billed tokens before indexing starts.

Shahbaz Khalid

Is it worth the cost?

Qu, Tu and Bao (Vectara and UW–Madison, 2024) compared semantic and fixed-size chunking on document retrieval, evidence retrieval and answer generation.

→ What they found

- The gains were task-dependent and inconsistent
- The extra compute was "not justified by consistent performance gains"
- Their paper: "Is Semantic Chunking Worth the Computational Cost?" (arXiv 2410.13070)

Before you switch, run both chunkers on your own questions and compare what gets retrieved.

Shahbaz Khalid

Mind the package

`SemanticChunker` lives in `langchain-experimental`. Version 0.4.2 warns on import:

```
# DeprecationWarning:
# `langchain-experimental` is being sunset
# and is no longer actively maintained.
```

- Pin the version, or keep a copy of the code you use.
- LlamaIndex's `SemanticSplitterNodeParser` works the same way, with the same defaults: `buffer_size=1`, 95th percentile.

Shahbaz Khalid

Put it all together

Cut only on outlier jumps, then cap the size on sentence boundaries:

```
semantic = SemanticChunker(
    embeddings,
    breakpoint_threshold_type="standard_deviation",
)
cap = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=0,
    separators=["\n\n", "\n", ". ", " ", ""],
    keep_separator="end",
)
chunks = [piece for c in semantic.split_text(text)
          for piece in cap.split_text(c)]
```

- 500 is an example. Size the cap to your embedder's input limit.
- Test it on a document with known topic changes before you trust the cuts.

Shahbaz Khalid

When it pays off

✓ Use it when

- Documents mix topics without headings: notes, merged pages, long emails
- Ingest volume is small enough to embed the text about 3 times
- It beats your current splitter on your own questions

→ Skip it when

- Documents have headings or clear structure: split on those
- A large corpus gets re-ingested often
- The text has no sentence punctuation, like raw transcripts

Shahbaz Khalid

RAG chunking **Semantic chunking**

Cheat sheet

- Embeds sentence groups, cuts at big distance jumps
- Default: the top 5% of gaps (95th percentile)
- One-topic docs still get cut: try `standard_deviation`
- No max size: cap it with a recursive splitter
- Sentences come from a regex: `e.g.` splits too
- About 3x your text embedded at ingest
- Compare it with fixed-size on your own questions

From the RAG series by Shahbaz Khalid

Shahbaz Khalid

Tomorrow →

RAG chunking

Small-to-big ▢▢ Part 1 of 2

Parent document retriever

Match on small chunks, then hand the LLM the whole section they came from.

✓ **Today: Semantic chunking**